

Architecture API GeniusPay V3 dans un Écosystème Next.js

Auteur : Solution Architect AI Assistant

Version de l'API : V3 (Dernière version stable — 2026)

Framework cible : Next.js 14/15 (App Router & Route Handlers)

Statut du Document : Prêt pour Implémentation Équipe Tech

Résumé Exécutif : Ce document technique définit de bout en bout le processus d'interconnexion avec la passerelle de paiement unifiée **GeniusPay Afrique V3**. Il couvre la cartographie complète de l'architecture RESTful, la gestion synchrone/asynchrone des transactions via les Route Handlers de Next.js, la sécurisation des processus d'arrière-plan via les Webhooks, ainsi que l'implémentation d'une cinétique de paiement fluide *sans redirection externe* (Direct Mode inline via API).

1. Architecture RESTful & Environnements

L'API GeniusPay V3 est entièrement structurée selon les standards REST. Toutes les requêtes entrantes doivent utiliser le protocole HTTPS sécurisé, inclure des payloads au format JSON et s'authentifier par le biais de headers spécifiques transmis de serveur à serveur.

Points de Terminaison Globaux (Base URLs)

Environnement	Base URL de l'API Marchand V3	Usage
Sandbox (Test)	https://labpay.genius.ci/api/v1/merchant	Simulations de paiements, tokens fictifs sans impact financier.
Production	https://pay.genius.ci/api/v1/merchant	Transactions réelles multi-opérateurs à travers les 24 pays.

En-têtes HTTP Requis pour l'Authentification

Toutes les communications vers l'API nécessitent l'usage conjoint de vos identifiants marchands. Contrairement aux anciennes versions utilisant l'en-tête *Bearer* générique pour certaines routes, le standard actuel de sécurité GeniusPay V3 impose de passer explicitement votre clé publique et votre clé secrète via des en-têtes personnalisés :

- **X-API-Key** : Clé publique marchande (ex: pk_live_... ou pk_test_...)
- **X-API-Secret** : Clé privée secrète marchande (ex: sk_live_... ou sk_test_...)
- **Content-Type** : Doit être systématiquement configuré à application/json

2. Répertoire des Endpoints & États des Transactions

Le cycle de vie d'un flux de paiement ou de transfert (Payout) repose sur les trois segments d'endpoints cruciaux décrits ci-dessous.

Méthode & Endpoint	Description Fonctionnelle
POST /payments	Initialise une intention de paiement ou exécute un débit direct (Mobile Money local ou Carte via Stripe).
GET /payments/{id}	Récupère l'état d'un paiement en temps réel (Utile pour le polling de secours côté client).
POST /payouts	Déclenche un transfert de fonds sortant depuis le compte marchand vers un compte Mobile Money client (ex: Salaires, Retraits).

Matrice des Statuts de Transaction

Lors de l'interrogation ou de la réception de notifications, la propriété status de la transaction adopte l'une des valeurs standardisées suivantes :

Statut API	Signification et Action requise par l'application Next.js
pending	La transaction est initiée. En attente que l'utilisateur saisisse son code PIN secret sur son téléphone (Mobile Money USSD / Push).
success	Les fonds ont été prélevés et sécurisés avec succès. Vous pouvez délivrer le produit/service au client de manière synchrone.
failed	La transaction a échoué. Causes : provision insuffisante, refus de l'opérateur, ou expiration du délai d'approbation (Timeout).

3. Intégration Directe (Sans Redirection Externe)

Oui, il est parfaitement possible d'intégrer GeniusPay sans rediriger l'utilisateur vers une page externe de type Checkout.

Le Mécanisme : Par défaut, si vous omettez le paramètre de méthode, l'API génère une URL de redirection générique. En revanche, si vous fournissez l'objet customer contenant le numéro de téléphone au format international combiné au paramètre payment_method (et optionnellement l'opérateur explicite mmo_provider), GeniusPay V3 déclenche un **débit push direct (USSD Push Notification)**.

L'utilisateur voit apparaître un pop-up natif directement sur l'écran de son smartphone lui demandant de valider la transaction en saisissant son code PIN secret. Le parcours utilisateur s'effectue intégralement en arrière-plan pendant qu'il reste sur votre interface Next.js.

Exemple de Payload JSON pour un Paiement Direct Mobile Money

```
{ "amount": 5000, "currency": "XOF", "payment_method": "pawapay", "mmo_provider":
"ORANGE_CIV", "customer": { "phone": "+2250700000000", "country": "CI" } }
```

4. Implémentation Pratique dans Next.js (App Router)

Pour des raisons de sécurité évidentes, les clés privées (X-API-Secret) ne doivent jamais être exposées au navigateur du client. Nous tirons parti des **Route Handlers** de Next.js pour concevoir une API intermédiaire ultra-sécurisée.

Fichier : app/api/checkout/route.js (Initialisation & Exécution)

```
import { NextResponse } from 'next/server'; export async function POST(request) { try {
const body = await request.json(); const { amount, phone, provider } = body; // Envoi de
la requête sécurisée de serveur à serveur vers GeniusPay const response = await
fetch('https://pay.genius.ci/api/v1/merchant/payments', { method: 'POST', headers: {
'X-API-Key': process.env.GENIUSPAY_PUBLIC_KEY, 'X-API-Secret':
process.env.GENIUSPAY_SECRET_KEY, 'Content-Type': 'application/json', }, body:
JSON.stringify({ amount: amount, currency: "XOF", payment_method: "pawapay",
mmo_provider: provider, // ex: 'WAVE', 'ORANGE_CIV', 'MTN_CIV' customer: { phone: phone,
country: "CI" } } }, {}); const data = await response.json(); if (!response.ok) { return
NextResponse.json({ error: data.message || 'Échec de la transaction' }, { status:
response.status }); } // Retourne le statut 'pending' et l'identifiant pour suivi return
NextResponse.json({ success: true, payment: data }); } catch (error) { return
NextResponse.json({ error: 'Internal Server Error' }, { status: 500 }); } }
```

5. Traitement Sécurisé des Webhooks

Parce que les transactions Mobile Money dépendent de la validation humaine de l'utilisateur sur son téléphone, la réponse initiale de l'API sera presque toujours pending. Pour confirmer le succès réel d'un paiement de manière asynchrone et infalsifiable, vous devez obligatoirement implémenter un endpoint Webhook.

GeniusPay pousse une notification HTTP POST vers l'URL que vous configurez sur votre tableau de bord (ex: <https://votredomaine.com/api/webhooks/geniuspay>) dès le changement de statut final.

Fichier : `app/api/webhooks/geniuspay/route.js`

```
import { NextResponse } from 'next/server'; import crypto from 'crypto'; export async
function POST(request) { try { const rawBody = await request.text(); const signature =
request.headers.get('x-geniuspay-signature'); const webhookSecret =
process.env.GENIUSPAY_WEBHOOK_SECRET; // Validation stricte de l'empreinte de sécurité du
Webhook const computedSignature = crypto .createHmac('sha256', webhookSecret)
.update(rawBody) .digest('hex'); if (signature !== computedSignature) { return
NextResponse.json({ error: 'Signature invalide' }, { status: 401 }); } const event =
JSON.parse(rawBody); // Routage logique selon la nature de l'événement reçu if
(event.type === 'payment.success') { const paymentData = event.data; const transactionId
= paymentData.id; // ACTION : Mettre à jour votre base de données (ex: Marquer la commande
comme PAYÉE) console.log(`Paiement validé pour la transaction: ${transactionId}`); } //
Répondre impérativement avec un statut 200 à GeniusPay pour accuser réception return
NextResponse.json({ received: true }, { status: 200 }); } catch (error) { return
NextResponse.json({ error: 'Webhook Handler Failed' }, { status: 500 }); } }
```

Note : Cette documentation constitue un guide technique d'intégration basé sur l'état de l'art de l'écosystème d'API GeniusPay Afrique V3. Veuillez systématiquement confronter ces implémentations à vos identifiants réels de bac à sable (Sandbox) avant tout déploiement en production.